МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу «Дискретный анализ»
на тему «Сортировки за линейное время»

Выполнил:	Н. В. Храмов
Группа:	М8О-208БВ-24
Преподаватель:	Макаров Н.К.
Проверил:	Сахарин Н.А.
Дата:	28.03.2026
Оценка:	
Подпись:	

Москва, 2025

Постановка задачи

Требуется разработать программу, осуществляющую ввод пар ключ-значение, их упорядочивание по возрастанию ключа указанным алгоритмом сортировки за линейное время и вывод отсортированной последовательности.

Вариант задания: Поразрядная сортировка.

Тип ключа: целые числа от 0 до 18446744073709551615 (2 в 64 степени минус 1).

Тип значения: целые числа в том же диапазоне.

Формат ввода

Каждая строка содержит ключ, символ табуляции и значение.

Формат вывода

Те же строки, отсортированные по ключу.

Алгоритм решения задачи

1. Организация данных

   Программа считывает ключи и значения в цикле, используя стандартный ввод. Данные помещаются в структуру Pair, которая затем сохраняется в динамический массив (вектор).

2. Подготовка к сортировке

   Создается вспомогательный массив (буфер) такого же размера, как и основной. Это необходимо для временного хранения данных при обработке каждого разряда.

3. Процесс поразрядной сортировки (Radix Sort)

   Так как ключ занимает 64 бита, мы разбиваем его на 8 частей по 8 бит (каждая часть — это один байт). Сортировка проходит в несколько этапов:

   -- На каждом этапе мы смотрим на конкретный байт числа, начиная с самого младшего.

   -- Подсчитываем, сколько раз встречается каждое значение байта (от 0 до 255).

-- Вычисляем позиции, в которые должны встать элементы в новом массиве.

-- Переносим элементы в буфер в правильном порядке.

-- После каждого этапа буфер и основной массив меняются ролями.

4. Результат

После 8 проходов данные оказываются полностью отсортированы. Программа выводит их в консоль.

Анализ сложности

Временная сложность:

-- Чтение и вывод занимают время, пропорциональное количеству элементов (n).

-- Сортировка делает ровно 8 проходов по массиву. Каждый проход занимает время, пропорциональное n.

-- Итоговая сложность составляет O(n), что является линейным временем.

Пространственная сложность:

-- Требуется память для хранения всех элементов и такая же область памяти для буфера.

Итоговая сложность по памяти O(n).

Исходный код

```
#include <iostream>
#include <cstddef>
#include <algorithm>

template <typename T>
class Vector
{
private:
    T *data;
    size_t sz;
    size_t capacity;

    void reallocate(size_t new_capacity)
```

```
    {
        T *new_data = new T[new_capacity];
        for (size_t i = 0; i < sz; ++i)
        {
            new_data[i] = data[i];
        }
        delete[] data;
        data = new_data;
        capacity = new_capacity;
    }

public:
    Vector() : data(nullptr), sz(0), capacity(0) {}

    explicit Vector(size_t n) : sz(n), capacity(n)
    {
        data = new T[n];
    }

    ~Vector() { delete[] data; }

    Vector(const Vector &other) : sz(other.sz), capacity(other.capacity)
    {
        data = new T[capacity];
        for (size_t i = 0; i < sz; ++i)
            data[i] = other.data[i];
    }

    Vector &operator=(const Vector &other)
    {
        if (this != &other)
        {
            delete[] data;
            sz = other.sz;
            capacity = other.capacity;
            data = new T[capacity];
            for (size_t i = 0; i < sz; ++i)
                data[i] = other.data[i];
        }
        return *this;
    }
```

```
    void push_back(const T &value)
    {
        if (sz == capacity)
        {
            reallocate(capacity == 0 ? 8 : capacity * 2);
        }
        data[sz++] = value;
    }

    size_t size() const { return sz; }
    T &operator[](size_t index) { return data[index]; }
    const T &operator[](size_t index) const { return data[index]; }

    T *begin() { return data; }
    T *end() { return data + sz; }
};

struct Pair
{
    unsigned long long key;
    unsigned long long value;
};

class RadixSorter
{
public:
    static void sort(Vector<Pair> &arr)
    {
        size_t n = arr.size();
        if (n <= 1)
            return;

        Vector<Pair> buffer(n);
        Vector<Pair> *src = &arr;
        Vector<Pair> *dst = &buffer;

        for (int shift = 0; shift < 64; shift += 8)
        {
            size_t count[256] = {0};

            for (size_t i = 0; i < n; ++i)
            {
```

```
                unsigned char byte = ((*src)[i].key >> shift) & 0xFF;
                count[byte]++;
            }

            size_t start_pos = 0;
            for (int i = 0; i < 256; ++i)
            {
                size_t c = count[i];
                count[i] = start_pos;
                start_pos += c;
            }

            for (size_t i = 0; i < n; ++i)
            {
                unsigned char byte = ((*src)[i].key >> shift) & 0xFF;
                (*dst)[count[byte]++] = (*src)[i];
            }

            std::swap(src, dst);
        }

        if (src != &arr)
        {
            for (size_t i = 0; i < n; ++i)
            {
                arr[i] = buffer[i];
            }
        }
    }
};

int main()
{
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    Vector<Pair> data;
    unsigned long long k, v;

    while (std::cin >> k >> v)
    {
        data.push_back({k, v});
```

```
    }

    RadixSorter::sort(data);

    for (size_t i = 0; i < data.size(); ++i)
    {
        std::cout << data[i].key << "\t" << data[i].value << "\n";
    }

    return 0;
}
```

Тесты

Тест 1

>>>

5 1530408634

2 999999999

8 10

1 5

<<<

1 5

2 999999999

5 1530408634

8 10

Тест 2

>>>

18446744073709551615 123456789

0 987654321

18446744073709551614 111111111

<<<

0 987654321

18446744073709551614 111111111

18446744073709551615 123456789

Тест 3

>>>

10 500

10 100

10 300

10 200

<<<

10 500

10 100

10 300

10 200

Тест 4

>>>

0 10

0 5

0 20

<<<

0 10

0 5

0 20

Вывод

Программа успешно выполняет сортировку за линейное время. Использование поразрядного метода позволяет эффективно обрабатывать 64-битные числа, что быстрее обычных методов сравнения.